```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

"""
cansat_sensors.py
Librería modular para la gestión de sensores del CanSat Team Rocket.
VERSIÓN DEFINITIVA CON TODAS LAS FUNCIONALIDADES:
- BME280: temperatura, presión, humedad, ALTITUD BAROMÉTRICA
- BNO055: aceleración, giroscopio, magnetómetro, Euler, CUATERNIONES
- GNSS: latitud, longitud, altitud GPS, velocidad, rumbo
- ADS1115: sensores analógicos
"""

import time
import logging
import struct
import math
from abc import ABC, abstractmethod
from datetime import datetime
from typing import Dict, Any, Optional, List

# Configuración de logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ------------------------------------------------------------------------------
# Importaciones específicas de sensores (con manejo de errores)
# ------------------------------------------------------------------------------
try:
    import board
    import busio
except ImportError:
    logger.error("Biblioteca 'board' o 'busio' no encontrada. Ejecuta: pip install adafruit-blinka")
    board = None
    busio = None

# BME280
try:
    from adafruit_bme280 import basic as adafruit_bme280
except ImportError:
    logger.error("Biblioteca BME280 no encontrada. Ejecuta: pip install adafruit-circuitpython-bme280")
    adafruit_bme280 = None

# ADS1115
try:
    from adafruit_ads1x15.ads1115 import ADS1115
    from adafruit_ads1x15.analog_in import AnalogIn
except ImportError:
    logger.error("Biblioteca ADS1x15 no encontrada. Ejecuta: pip install adafruit-circuitpython-ads1x15")
    ADS1115 = None
    AnalogIn = None

# GNSS (librería personalizada DFRobot)
try:
    from GNSSAndRTC import GNSSAndRTC
except ImportError:
    logger.error("Librería GNSSAndRTC no encontrada. Asegúrate de que GNSSAndRTC.py está en el mismo directorio.")
    GNSSAndRTC = None

# smbus2 para BNO055
try:
    import smbus2
except ImportError:
    logger.error("Biblioteca smbus2 no encontrada. Ejecuta: pip install smbus2")
```

```python
    smbus2 = None


# ---------------------------------------------------------------------------
# Implementación para BNO055 usando smbus2 (CON CUATERNIONES)
# ---------------------------------------------------------------------------
class BNO055_smbus:
    """
    Implementación del BNO055 usando smbus2 directamente.
    Incluye TODOS los datos: aceleración, giroscopio, magnetómetro,
    ángulos de Euler y CUATERNIONES.
    """
    # Registros importantes
    CHIP_ID_ADDR = 0x00
    OPR_MODE_ADDR = 0x3D
    PWR_MODE_ADDR = 0x3E
    SYS_TRIGGER_ADDR = 0x3F
    TEMP_ADDR = 0x34
    EULER_H_LSB = 0x1A
    ACCEL_DATA_X_LSB = 0x08
    GYRO_DATA_X_LSB = 0x14
    MAG_DATA_X_LSB = 0x0E
    CALIB_STAT_ADDR = 0x35
    QUATERNION_DATA_W_LSB = 0x20  # Registro para cuaterniones (4 valores de 2 bytes)

    # Modos de operación
    OPERATION_MODE_CONFIG = 0x00
    OPERATION_MODE_NDOF = 0x0C  # Modo que incluye cuaterniones

    # ID esperado del chip
    EXPECTED_CHIP_ID = 0xA0

    def __init__(self, bus_id=1, address=0x28):
        self.bus_id = bus_id
        self.address = address
        self.bus = None
        self._initialized = False

    def init(self):
        """Inicializa la comunicación con el BNO055."""
        if smbus2 is None:
            logger.error("BNO055: smbus2 no disponible")
            return False

        try:
            self.bus = smbus2.SMBus(self.bus_id)

            # Verificar chip ID
            chip_id = self._read_byte(self.CHIP_ID_ADDR)
            logger.info(f"BNO055: Chip ID = 0x{chip_id:02X}")

            if chip_id != self.EXPECTED_CHIP_ID:
                logger.error(f"BNO055: Chip ID incorrecto (esperado 0xA0, recibido 0x{chip_id:02X})")
                return False

            # Configurar modo CONFIG
            self._write_byte(self.OPR_MODE_ADDR, self.OPERATION_MODE_CONFIG)
            time.sleep(0.1)

            # Resetear sensor
            self._write_byte(self.SYS_TRIGGER_ADDR, 0x20)
            time.sleep(1)

            # Cambiar a modo NDOF (fusión de sensores) - INCLUYE CUATERNIONES
            self._write_byte(self.OPR_MODE_ADDR, self.OPERATION_MODE_NDOF)
            time.sleep(0.1)

            self._initialized = True
```

```python
            logger.info("BNO055: Inicializado correctamente (modo NDOF - con cuaterniones)")
            return True

        except Exception as e:
            logger.error(f"BNO055: Error en init: {e}")
            return False

    def _read_byte(self, reg):
        """Lee un byte del registro."""
        return self.bus.read_byte_data(self.address, reg)

    def _read_bytes(self, reg, length):
        """Lee múltiples bytes del registro."""
        return self.bus.read_i2c_block_data(self.address, reg, length)

    def _write_byte(self, reg, value):
        """Escribe un byte en el registro."""
        self.bus.write_byte_data(self.address, reg, value)

    @property
    def temperature(self):
        """Lee la temperatura en grados Celsius."""
        return self._read_byte(self.TEMP_ADDR)

    @property
    def quaternion(self):
        """
        Lee los cuaterniones (w, x, y, z).
        Los cuaterniones son la mejor representación para orientación 3D.
        """
        try:
            # Leer 8 bytes (4 valores de 2 bytes cada uno)
            data = self._read_bytes(self.QUATERNION_DATA_W_LSB, 8)

            # Los cuaterniones están en formato Q16 (dividir por 2^14)
            w = struct.unpack('<h', bytes(data[0:2]))[0] / 16384.0
            x = struct.unpack('<h', bytes(data[2:4]))[0] / 16384.0
            y = struct.unpack('<h', bytes(data[4:6]))[0] / 16384.0
            z = struct.unpack('<h', bytes(data[6:8]))[0] / 16384.0

            return (w, x, y, z)
        except Exception as e:
            logger.debug(f"Error leyendo cuaterniones: {e}")
            return (0.0, 0.0, 0.0, 0.0)

    @property
    def euler(self):
        """Lee los ángulos de Euler: heading, roll, pitch."""
        try:
            data = self._read_bytes(self.EULER_H_LSB, 6)
            heading = struct.unpack('<h', bytes(data[0:2]))[0] / 16.0
            roll = struct.unpack('<h', bytes(data[2:4]))[0] / 16.0
            pitch = struct.unpack('<h', bytes(data[4:6]))[0] / 16.0
            return (heading, roll, pitch)
        except:
            return (0.0, 0.0, 0.0)

    @property
    def acceleration(self):
        """Lee la aceleración en m/s²."""
        try:
            data = self._read_bytes(self.ACCEL_DATA_X_LSB, 6)
            x = struct.unpack('<h', bytes(data[0:2]))[0] / 100.0
            y = struct.unpack('<h', bytes(data[2:4]))[0] / 100.0
            z = struct.unpack('<h', bytes(data[4:6]))[0] / 100.0
            return (x, y, z)
        except:
            return (0.0, 0.0, 0.0)

```

```python
    @property
    def gyro(self):
        """Lee la velocidad angular en rad/s."""
        try:
            data = self._read_bytes(self.GYRO_DATA_X_LSB, 6)
            x = struct.unpack('<h', bytes(data[0:2]))[0] / 16.0
            y = struct.unpack('<h', bytes(data[2:4]))[0] / 16.0
            z = struct.unpack('<h', bytes(data[4:6]))[0] / 16.0
            return (x, y, z)
        except:
            return (0.0, 0.0, 0.0)

    @property
    def magnetic(self):
        """Lee el campo magnético en uT."""
        try:
            data = self._read_bytes(self.MAG_DATA_X_LSB, 6)
            x = struct.unpack('<h', bytes(data[0:2]))[0] / 16.0
            y = struct.unpack('<h', bytes(data[2:4]))[0] / 16.0
            z = struct.unpack('<h', bytes(data[4:6]))[0] / 16.0
            return (x, y, z)
        except:
            return (0.0, 0.0, 0.0)

    @property
    def calibration_status(self):
        """Lee el estado de calibración (sys, gyro, accel, mag)."""
        try:
            calib = self._read_byte(self.CALIB_STAT_ADDR)
            sys = (calib >> 6) & 0x03
            gyro = (calib >> 4) & 0x03
            accel = (calib >> 2) & 0x03
            mag = calib & 0x03
            return (sys, gyro, accel, mag)
        except:
            return (0, 0, 0, 0)


# ---------------------------------------------------------------------------
# Clase Base Abstracta (ABC) para todos los sensores
# ---------------------------------------------------------------------------
class SensorBase(ABC):
    """
    Clase base abstracta que define la interfaz común para todos los sensores.
    """
    def __init__(self, name: str):
        """
        Inicializa el sensor base.
        Args:
            name: Nombre descriptivo del sensor (para logging).
        """
        self.name = name
        self._last_read_time: Optional[datetime] = None
        self._last_data: Dict[str, Any] = {}

    @abstractmethod
    def init(self) -> bool:
        """
        Inicializa el sensor. Debe ser implementado por cada subclase.
        Returns:
            bool: True si la inicialización fue exitosa, False en caso contrario.
        """
        pass

    @abstractmethod
    def read(self) -> bool:
        """
        Realiza una lectura del sensor y almacena los datos internamente.
        Returns:
```

```
            bool: True si la lectura fue exitosa, False en caso contrario.
        """
        pass

    @abstractmethod
    def get_data(self) -> Dict[str, Any]:
        """
        Devuelve los datos leídos en el último ciclo.
        Returns:
            dict: Diccionario con los valores del sensor.
        """
        pass

    def _update_timestamp(self):
        """Actualiza el timestamp interno con la hora actual."""
        self._last_read_time = datetime.now()


# ---------------------------------------------------------------------------
# BME280: Temperatura, Presión, Humedad y ALTITUD BAROMÉTRICA
# ---------------------------------------------------------------------------
class BME280Sensor(SensorBase):
    """
    Sensor BME280 para temperatura, presión, humedad y ALTITUD BAROMÉTRICA.
    La altitud se calcula a partir de la presión atmosférica.
    """
    def __init__(self, i2c_bus, address: int = 0x76, sea_level_pressure: float = 1013.25):
        """
        Args:
            i2c_bus: Objeto busio.I2C.
            address: Dirección I2C del sensor (por defecto 0x76).
            sea_level_pressure: Presión a nivel del mar en hPa para cálculo de altitud.
        """
        super().__init__("BME280")
        self.i2c_bus = i2c_bus
        self.address = address
        self.sea_level_pressure = sea_level_pressure
        self.sensor = None
        self._data = {
            "temperature": 0.0,
            "pressure": 0.0,
            "humidity": 0.0,
            "altitude": 0.0  # Altitud por presión barométrica
        }

    def init(self) -> bool:
        """Inicializa el BME280."""
        if adafruit_bme280 is None:
            logger.error(f"{self.name}: Biblioteca no disponible.")
            return False

        # Probar ambas direcciones (0x76 y 0x77)
        direcciones = [0x76, 0x77]

        for addr in direcciones:
            try:
                logger.info(f"{self.name}: Probando dirección 0x{addr:02X}...")
                self.sensor = adafruit_bme280.Adafruit_BME280_I2C(self.i2c_bus, address=addr)

                # Configurar presión a nivel del mar
                self.sensor.sea_level_pressure = self.sea_level_pressure

                # Hacer una lectura de prueba
                temp = self.sensor.temperature
                press = self.sensor.pressure
                hum = self.sensor.humidity

                if temp is not None and press is not None and hum is not None:
                    logger.info(f"{self.name}: Inicializado correctamente en 0x{addr:02X}")
```

```python
                    logger.info(f"  Temp: {temp:.1f}°C, Presión: {press:.1f}hPa, Hum: {hum:.1f}%")
                    self.address = addr
                    return True

            except Exception as e:
                logger.debug(f"{self.name}: Error en 0x{addr:02X}: {e}")
                continue

        logger.error(f"{self.name}: No se pudo inicializar en ninguna dirección")
        return False

    def read(self) -> bool:
        """Realiza una lectura incluyendo altitud por presión."""
        if self.sensor is None:
            logger.warning(f"{self.name}: Sensor no inicializado.")
            return False

        try:
            # Leer datos básicos
            self._data["temperature"] = round(self.sensor.temperature, 2)
            self._data["pressure"] = round(self.sensor.pressure, 2)
            self._data["humidity"] = round(self.sensor.humidity, 2)

            # CALCULAR ALTITUD POR PRESIÓN (fórmula barométrica)
            pressure_hPa = self._data["pressure"]
            if pressure_hPa > 0:
                # altitud = 44330 * (1 - (P/P0)^(1/5.255))
                altitude = 44330.0 * (1.0 - math.pow(pressure_hPa / self.sea_level_pressure, 1/5.255))
                self._data["altitude"] = round(altitude, 1)
            else:
                self._data["altitude"] = 0.0

            self._update_timestamp()
            return True

        except Exception as e:
            logger.error(f"{self.name}: Error en lectura: {e}")
            return False

    def get_data(self) -> Dict[str, Any]:
        """Devuelve los datos incluyendo altitud barométrica."""
        return self._data.copy()

    def set_sea_level_pressure(self, pressure: float):
        """Actualiza la presión a nivel del mar para cálculos más precisos."""
        self.sea_level_pressure = pressure
        if self.sensor:
            self.sensor.sea_level_pressure = pressure
        logger.info(f"{self.name}: Presión nivel mar actualizada a {pressure:.1f}hPa")


# ---------------------------------------------------------------------------
# BNO055: Aceleración, Giroscopio, Magnetómetro, Euler y CUATERNIONES
# ---------------------------------------------------------------------------
class BNO055Sensor(SensorBase):
    """
    Sensor BNO055: aceleración, giroscopio, magnetómetro, orientación y CUATERNIONES.
    Implementación usando smbus2.
    """
    def __init__(self, bus_id: int = 1, address: int = 0x28):
        super().__init__("BNO055")
        self.bus_id = bus_id
        self.address = address
        self.sensor = None
        self._data = {
            # Aceleración
            "accel_x": 0.0, "accel_y": 0.0, "accel_z": 0.0,
```

```
            # Giroscopio
            "gyro_x": 0.0, "gyro_y": 0.0, "gyro_z": 0.0,
            # Magnetómetro
            "mag_x": 0.0, "mag_y": 0.0, "mag_z": 0.0,
            # Ángulos de Euler
            "euler_heading": 0.0, "euler_roll": 0.0, "euler_pitch": 0.0,
            # CUATERNIONES
            "quat_w": 0.0, "quat_x": 0.0, "quat_y": 0.0, "quat_z": 0.0,
            # Temperatura
            "temperature": 0.0,
            # Calibración
            "calibration_sys": 0,
            "calibration_gyro": 0,
            "calibration_accel": 0,
            "calibration_mag": 0
        }

    def init(self) -> bool:
        """Inicializa el BNO055 usando la implementación smbus2."""
        try:
            self.sensor = BNO055_smbus(bus_id=self.bus_id, address=self.address)
            if self.sensor.init():
                logger.info(f"{self.name}: Inicializado correctamente (con cuaterniones)")
                return True
            else:
                logger.error(f"{self.name}: Falló la inicialización")
                return False
        except Exception as e:
            logger.error(f"{self.name}: Error en init: {e}")
            return False

    def read(self) -> bool:
        """Realiza una lectura de TODOS los datos del BNO055, incluyendo cuaterniones."""
        if self.sensor is None:
            logger.warning(f"{self.name}: Sensor no inicializado.")
            return False
        try:
            # Aceleración
            accel = self.sensor.acceleration
            if accel:
                self._data["accel_x"], self._data["accel_y"], self._data["accel_z"] = [round(v, 3) for v in accel]

            # Giroscopio
            gyro = self.sensor.gyro
            if gyro:
                self._data["gyro_x"], self._data["gyro_y"], self._data["gyro_z"] = [round(v, 3) for v in gyro]

            # Magnetómetro
            mag = self.sensor.magnetic
            if mag:
                self._data["mag_x"], self._data["mag_y"], self._data["mag_z"] = [round(v, 3) for v in mag]

            # Euler
            euler = self.sensor.euler
            if euler:
                self._data["euler_heading"], self._data["euler_roll"], self._data["euler_pitch"] = [round(v, 2) for v in euler]

            # CUATERNIONES
            quat = self.sensor.quaternion
            if quat:
                self._data["quat_w"], self._data["quat_x"], self._data["quat_y"], self._data["quat_z"] = [round(v, 4) for v in quat]

            # Temperatura
            temp = self.sensor.temperature
```

```python
            if temp is not None:
                self._data["temperature"] = round(temp, 1)

            # Calibración
            calib = self.sensor.calibration_status
            if calib:
                self._data["calibration_sys"], self._data["calibration_gyro"], self._data["calibration_accel"], self._data["calibration_mag"] = calib

            self._update_timestamp()
            return True
        except Exception as e:
            logger.error(f"{self.name}: Error en lectura: {e}")
            return False

    def get_data(self) -> Dict[str, Any]:
        """Devuelve un diccionario con TODOS los datos del BNO055."""
        return self._data.copy()


# ---------------------------------------------------------------------------
# GNSS: Latitud, Longitud, Altitud GPS, Velocidad, Rumbo
# ---------------------------------------------------------------------------
class GNSSSensor(SensorBase):
    """
    Módulo GNSS+RTC DFR1103.
    Proporciona latitud, longitud, altitud GPS, velocidad, rumbo y tiempo.
    """
    def __init__(self, bus_id: int = 1, address: int = 0x66):
        super().__init__("GNSS")
        self.bus_id = bus_id
        self.address = address
        self.sensor = None
        self._data = {
            "latitude": 0.0,
            "longitude": 0.0,
            "altitude": 0.0,
            "speed": 0.0,
            "course": 0.0,
            "satellites": 0,
            "year": 0,
            "month": 0,
            "day": 0,
            "hour": 0,
            "minute": 0,
            "second": 0,
            "fix_valid": False
        }

    def init(self) -> bool:
        """Inicializa el módulo GNSS."""
        if GNSSAndRTC is None:
            logger.error(f"{self.name}: Librería no disponible.")
            return False
        try:
            self.sensor = GNSSAndRTC(bus=self.bus_id, addr=self.address)
            # Verificar comunicación
            if not self.sensor.detectar():
                logger.error(f"{self.name}: Módulo no detectado en la dirección {hex(self.address)}.")
                return False
            logger.info(f"{self.name}: Módulo detectado correctamente.")
            return True
        except Exception as e:
            logger.error(f"{self.name}: Error en init: {e}")
            return False

    def read(self) -> bool:
        """Realiza una lectura de TODOS los datos GNSS."""
```

```python
        if self.sensor is None:
            logger.warning(f"{self.name}: Sensor no inicializado.")
            return False
        try:
            # Usar métodos individuales para más control
            año, mes, dia = self.sensor.get_fecha()
            hora, minuto, segundo = self.sensor.get_hora_utc()
            latitud = self.sensor.get_latitud()
            longitud = self.sensor.get_longitud()
            altitud = self.sensor.get_altitud()
            satelites = self.sensor.get_satelites()

            # Intentar obtener velocidad y rumbo
            velocidad = 0.0
            rumbo = 0.0
            if hasattr(self.sensor, 'get_velocidad'):
                velocidad = self.sensor.get_velocidad()
            if hasattr(self.sensor, 'get_rumbo'):
                rumbo = self.sensor.get_rumbo()

            # Verificar fix válido
            fix_valido = (año > 2020 and satelites >= 3 and abs(latitud) > 0.0001 and abs(longitud) > 0.0001)

            # Actualizar datos
            self._data.update({
                "latitude": latitud,
                "longitude": longitud,
                "altitude": altitud,
                "speed": velocidad,
                "course": rumbo,
                "satellites": satelites,
                "year": año,
                "month": mes,
                "day": dia,
                "hour": hora,
                "minute": minuto,
                "second": segundo,
                "fix_valid": fix_valido
            })

            self._update_timestamp()
            return True

        except Exception as e:
            logger.error(f"{self.name}: Error en lectura: {e}")
            return False

    def get_data(self) -> Dict[str, Any]:
        """Devuelve los datos GNSS completos."""
        return self._data.copy()


# ---------------------------------------------------------------------------
# Sensor Analógico (ADS1115)
# ---------------------------------------------------------------------------
class AnalogSensor(SensorBase):
    """
    Sensor analógico conectado a través de un ADS1115.
    """
    def __init__(self, i2c_bus, channel: int, name: str = "Analog", ads_address: int = 0x48, gain: int = 1):
        """
        Args:
            i2c_bus: Objeto busio.I2C.
            channel: Canal del ADS1115 (0-3).
            name: Nombre personalizado para este sensor.
            ads_address: Dirección I2C del ADS1115.
            gain: Ganancia del ADS1115.
```

```python
        """
        super().__init__(name)
        self.i2c_bus = i2c_bus
        self.channel = channel
        self.ads_address = ads_address
        self.gain = gain
        self.ads = None
        self.analog_in = None
        self._data = {
            "value": 0,
            "voltage": 0.0
        }

    def init(self) -> bool:
        """Inicializa el ADS1115 y el canal analógico."""
        if ADS1115 is None or AnalogIn is None:
            logger.error(f"{self.name}: Biblioteca ADS1x15 no disponible.")
            return False
        try:
            self.ads = ADS1115(self.i2c_bus, address=self.ads_address)
            self.ads.gain = self.gain
            self.analog_in = AnalogIn(self.ads, self.channel)
            logger.info(f"{self.name} (canal {self.channel}): Inicializado correctamente.")
            return True
        except Exception as e:
            logger.error(f"{self.name}: Error en init: {e}")
            return False

    def read(self) -> bool:
        """Realiza una lectura del canal analógico."""
        if self.analog_in is None:
            logger.warning(f"{self.name}: Canal no inicializado.")
            return False
        try:
            self._data["value"] = self.analog_in.value
            self._data["voltage"] = round(self.analog_in.voltage, 3)
            self._update_timestamp()
            return True
        except Exception as e:
            logger.error(f"{self.name}: Error en lectura: {e}")
            return False

    def get_data(self) -> Dict[str, Any]:
        """Devuelve el valor raw y el voltaje."""
        return self._data.copy()


# ---------------------------------------------------------------------------
# SensorManager: Clase principal que gestiona TODOS los sensores
# ---------------------------------------------------------------------------
class SensorManager:
    """
    Gestiona todos los sensores del CanSat.
    Proporciona métodos para inicializar y leer todos los sensores de una sola vez.
    """
    def __init__(self, use_bme280: bool = True, use_bno055: bool = True,
                 use_gnss: bool = True, use_analog: bool = True,
                 analog_channels: Optional[List[int]] = None,
                 sea_level_pressure: float = 1013.25):
        """
        Args:
            use_bme280: Activar sensor BME280.
            use_bno055: Activar sensor BNO055.
            use_gnss: Activar módulo GNSS.
            use_analog: Activar sensores analógicos.
            analog_channels: Lista de canales analógicos [0, 1, 2, 3].
            sea_level_pressure: Presión a nivel del mar para altitud barométrica.
        """
        self.use_bme280 = use_bme280
```

```python
        self.use_bno055 = use_bno055
        self.use_gnss = use_gnss
        self.use_analog = use_analog
        self.analog_channels = analog_channels if analog_channels else [0]
        self.sea_level_pressure = sea_level_pressure

        self.sensors: Dict[str, SensorBase] = {}
        self.i2c = None
        self._initialized = False

    def init_all(self) -> bool:
        """
        Inicializa el bus I2C y TODOS los sensores configurados.
        Returns:
            bool: True si al menos un sensor se inicializó correctamente.
        """
        logger.info("Inicializando bus I2C...")
        if board is None or busio is None:
            logger.error("Bibliotecas de placa no disponibles. No se puede continuar.")
            return False

        try:
            self.i2c = busio.I2C(board.SCL, board.SDA)
            time.sleep(0.1)  # Estabilizar bus
        except Exception as e:
            logger.error(f"No se pudo inicializar el bus I2C: {e}")
            return False

        success = False

        # BME280
        if self.use_bme280:
            for addr in [0x76, 0x77]:
                sensor = BME280Sensor(self.i2c, address=addr, sea_level_pressure=self.sea_level_pressure)
                if sensor.init():
                    self.sensors["bme280"] = sensor
                    success = True
                    break
            if "bme280" not in self.sensors:
                logger.warning("BME280 no disponible, se omitirá.")

        # BNO055
        if self.use_bno055:
            for addr in [0x28, 0x29]:
                sensor = BNO055Sensor(bus_id=1, address=addr)
                if sensor.init():
                    self.sensors["bno055"] = sensor
                    success = True
                    break
            if "bno055" not in self.sensors:
                logger.warning("BNO055 no disponible, se omitirá.")

        # GNSS
        if self.use_gnss:
            sensor = GNSSSensor(bus_id=1, address=0x66)
            if sensor.init():
                self.sensors["gnss"] = sensor
                success = True
            else:
                logger.warning("GNSS no disponible, se omitirá.")

        # Sensores Analógicos
        if self.use_analog:
            for ch in self.analog_channels:
                sensor_name = f"analog_ch{ch}"
                sensor = AnalogSensor(self.i2c, channel=ch, name=sensor_name, ads_address=0x48)
                if sensor.init():
                    self.sensors[sensor_name] = sensor
```

```
                    success = True
                else:
                    logger.warning(f"{sensor_name} no disponible, se omitirá.")

        self._initialized = success
        if not success:
            logger.error("Ningún sensor pudo ser inicializado.")
        else:
            logger.info(f"Total de sensores activos: {len(self.sensors)}")
            for name in self.sensors.keys():
                logger.info(f"  - {name}")

        return success

    def read_all(self) -> Dict[str, Any]:
        """
        Lee TODOS los sensores activos y combina sus datos en un único diccionario.
        Returns:
            dict: Diccionario con timestamp y todos los datos de los sensores.
        """
        if not self._initialized:
            logger.warning("SensorManager no inicializado. Llama a init_all() primero.")
            return {"timestamp": datetime.now().isoformat(timespec='seconds'), "error": "Not initialized"}

        combined_data = {
            "timestamp": datetime.now().isoformat(timespec='seconds')
        }

        for name, sensor in self.sensors.items():
            try:
                if sensor.read():
                    data = sensor.get_data()

                    if name == "bme280":
                        combined_data.update({
                            "temperature": data["temperature"],
                            "pressure": data["pressure"],
                            "humidity": data["humidity"],
                            "altitude_barometric": data["altitude"]  # Altitud por presión
                        })

                    elif name == "bno055":
                        # Añadir todos los datos del BNO055 con prefijo
                        for key, value in data.items():
                            combined_data[f"bno_{key}"] = value

                    elif name == "gnss":
                        combined_data.update({
                            "latitude": data["latitude"],
                            "longitude": data["longitude"],
                            "gps_altitude": data["altitude"],
                            "gps_speed": data["speed"],
                            "gps_speed_kmh": data["speed"] * 3.6,  # Velocidad en km/h
                            "gps_course": data["course"],
                            "satellites": data["satellites"],
                            "gps_time": f"{data['hour']:02d}:{data['minute']:02d}:{data['second']:02d}",
                            "gps_date": f"{data['year']:04d}-{data['month']:02d}-{data['day']:02d}",
                            "fix_valid": data["fix_valid"]
                        })

                    elif name.startswith("analog"):
                        combined_data[f"{name}_value"] = data["value"]
                        combined_data[f"{name}_voltage"] = data["voltage"]

            except Exception as e:
                logger.error(f"Error procesando datos de {name}: {e}")
```

```python

        return combined_data

    def get_sensor(self, name: str) -> Optional[SensorBase]:
        """Devuelve la instancia de un sensor por su nombre."""
        return self.sensors.get(name)

    def get_sensor_names(self) -> List[str]:
        """Devuelve la lista de nombres de sensores activos."""
        return list(self.sensors.keys())

    def is_initialized(self) -> bool:
        """Indica si el gestor está inicializado."""
        return self._initialized

    def shutdown(self):
        """Libera recursos."""
        logger.info("Apagando sensores...")
        self.sensors.clear()
        self._initialized = False


# ---------------------------------------------------------------------------
# Funciones de utilidad
# ---------------------------------------------------------------------------
def create_default_sensor_manager(sea_level_pressure: float = 1013.25) -> SensorManager:
    """
    Crea un SensorManager con la configuración por defecto para CanSat.
    Incluye BME280, BNO055, GNSS y dos canales analógicos.
    """
    return SensorManager(
        use_bme280=True,
        use_bno055=True,
        use_gnss=True,
        use_analog=True,
        analog_channels=[0, 1],
        sea_level_pressure=sea_level_pressure
    )


def scan_i2c_devices() -> List[int]:
    """
    Escanea el bus I2C en busca de dispositivos.
    Returns:
        list: Lista de direcciones I2C encontradas.
    """
    if board is None or busio is None:
        logger.error("Bibliotecas de placa no disponibles.")
        return []

    try:
        i2c = busio.I2C(board.SCL, board.SDA)
        devices = []
        while not i2c.try_lock():
            pass
        try:
            for address in range(0x08, 0x78):
                if i2c.probe(address):
                    devices.append(address)
                    logger.info(f"Dispositivo I2C encontrado en: 0x{address:02X}")
        finally:
            i2c.unlock()
        return devices
    except Exception as e:
        logger.error(f"Error escaneando I2C: {e}")
        return []
```